

Week 13 easier

Week 13: Introduction to PyTorch. Tensors, automatic differentiation, and building basic Multilayer Perceptrons (MLPs) for signal classification (Raschka).

Can you map out two 75 min lectures for week 13. The class is a senior/graduate level DSP and ML course. The textbooks are Oppenheim/Schafer "Discrete-Time Signal Processing" and Raschka "Machine Learning with PyTorch and Scikit-Learn"

Gemini Pro:

Here is a structured plan for Week 13, bridging the deterministic signal processing of Oppenheim/Schafer with the data-driven framework of Raschka.

Lecture 25: Tensors and Automatic Differentiation (75 Min)

This session transitions students from traditional DSP arrays to the computational graphs that power modern machine learning.

- **From Vectors to Tensors (20 min):** Map 1D discrete-time signals $x[n]$ and 2D spectrograms to PyTorch tensors. Contrast standard NumPy arrays with PyTorch tensors, emphasizing GPU acceleration and gradient tracking.
- **Tensor Operations for DSP (20 min):** Demonstrate matrix multiplication, broadcasting, and dimensionality manipulation. Explain how a batch of framed audio is typically represented as a 3D tensor: (N, C, L) for Batch, Channel, and Length.
- **The Chain Rule & Autograd (35 min):** Introduce `torch.autograd`. Explain how PyTorch dynamically builds a directed acyclic graph during the forward pass to automatically compute gradients $\nabla_{\theta} L$ via backpropagation, eliminating the need to derive complex partial derivatives by hand.

Lecture 26: MLPs for Signal Classification (75 Min)

This session focuses on designing, assembling, and training a neural network to classify extracted DSP features (e.g., MFCCs or spectral flux).

- **Architecture of an MLP (20 min):** Define the linear algebraic forward pass $\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$. Discuss hidden layers and the critical necessity of non-linear activations (like ReLU) to draw complex decision boundaries between signal classes.
- **Loss Functions & Optimizers (25 min):** Connect the network's predictions to a measurable cost. Introduce Cross-Entropy Loss for multi-class signal classification, and explain how optimizers like Adam update the network's weights: $\theta_{t+1} = \theta_t - \eta \nabla L$.

- **The PyTorch Training Loop (30 min):** Walk through the standard 5-step training idiom required in PyTorch:
 1. Pass the signal features forward.
 2. Compute the loss.
 3. Clear old gradients with `optimizer.zero_grad()`.
 4. Backpropagate the error with `loss.backward()`.
 5. Update weights with `optimizer.step()`.
-

Week 13, Lecture 25: Tensors and Automatic Differentiation (75 Min)

This session transitions students from traditional DSP arrays to the computational graphs that power modern machine learning.

- **From Vectors to Tensors (20 min):** Map 1D discrete-time signals $x[n]$ and 2D spectrograms to PyTorch tensors. Contrast standard NumPy arrays with PyTorch tensors, emphasizing GPU acceleration and gradient tracking.
- **Tensor Operations for DSP (20 min):** Demonstrate matrix multiplication, broadcasting, and dimensionality manipulation. Explain how a batch of framed audio is typically represented as a 3D tensor: (N, C, L) for Batch, Channel, and Length.
- **The Chain Rule & Autograd (35 min):** Introduce `torch.autograd`. Explain how PyTorch dynamically builds a directed acyclic graph during the forward pass to automatically compute gradients $\nabla_{\theta} L$ via backpropagation, eliminating the need to derive complex partial derivatives by hand.

Prompt: Can you create slides and python code for this 75 minute class of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schaffer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well. Please render math equations in LaTeX.

ChatGPT:

I'll make this lecture explicitly bridge three things the students already know: $x[n]$ as a discrete-time sequence, framed/STFT audio as multidimensional arrays, and the chain rule from calculus. The central DSP example will be a small learnable FIR system so autograd feels like a generalization of familiar convolution—not a black-box neural-network trick.

ChatGPT: (Medium)

Created. The package is built around the DSP-to-ML transition you described, with tensor shapes tied directly to signals, spectrograms, filterbanks, and learnable FIR filters.

Files

- [Complete Week 13 Lecture 25 instructor package](#)
- [36-slide PowerPoint](#)
- [Jupyter notebook](#)
- [Executed/verified notebook](#)
- [Standalone Python code](#)
- [Homework, quiz, and worked solutions](#)
- [README / teaching flow](#)

The notebook was executed successfully with **no errors**, and the PowerPoint contains **36 slides**.

Lecture structure

The deck follows the requested 75-minute flow:

0–20 min — From vectors to tensors

Students move from familiar DSP notation,

$$x[n]$$

to tensors such as

$$\mathbf{x} \in \mathbb{R}^L$$

for a waveform and

$$\mathbf{X} \in \mathbb{C}^{F \times T}$$

for an STFT. The slides compare NumPy and PyTorch and emphasize that tensors add device placement and optional gradient tracking rather than changing the underlying DSP mathematics.

20–40 min — Tensor operations for DSP

The lecture develops:

$$(N, C, L)$$

for batch, channel, and signal length, along with reshaping, unsqueeze, broadcasting, and matrix multiplication.

One DSP-specific example treats a filterbank as

$$\mathbf{E} = \mathbf{W}\mathbf{P},$$

where

$$\mathbf{P} \in \mathbb{R}^{F \times T}, \quad \mathbf{W} \in \mathbb{R}^{M \times F},$$

so that

$$\mathbf{E} \in \mathbb{R}^{M \times T}.$$

This helps students see that tensor operations are not foreign ML constructs—they are familiar DSP linear algebra with more explicit dimensions.

40–75 min — Chain rule and autograd

The deck starts with a scalar chain-rule calculation by hand and then maps it to a computational graph.

For example,

$$\begin{aligned} u &= 2\theta - 3, \\ L &= (u + 1)^2, \end{aligned}$$

gives

$$\frac{dL}{d\theta} = \frac{dL}{du} \frac{du}{d\theta}.$$

Then PyTorch performs the same calculation with

```
theta = torch.tensor(4.0, requires_grad=True)
```

```
u = 2 * theta - 3
L = (u + 1)**2
```

```
L.backward()
```

```
print(theta.grad)
```

The main DSP example treats FIR coefficients as learnable parameters:

$$\hat{y}[n] = \sum_{k=0}^{M-1} h[k]x[n-k].$$

Using the mean-square loss

$$L = \frac{1}{N} \sum_n (\hat{y}[n] - y[n])^2,$$

the analytical gradient is

$$\frac{\partial L}{\partial h[r]} = \frac{2}{N} \sum_n (\hat{y}[n] - y[n]) x[n-r].$$

The lecture then shows that autograd produces this gradient automatically and uses gradient descent,

$$h_r^{(m+1)} = h_r^{(m)} - \eta \frac{\partial L}{\partial h[r]},$$

to recover an unknown FIR filter.

The demonstration converges from zero taps toward approximately

$$[0.15, ; 0.50, ; 0.25, ; -0.10]$$

so students see “training” as something directly connected to DSP system identification rather than immediately jumping into a neural-network black box.

Homework and quiz

The package includes **7 homework problems** covering tensor shapes, broadcasting, matrix multiplication, the chain rule, gradient accumulation, learnable FIR gradients, and `detach()` versus `no_grad()`.

There is also a **10-minute weekly quiz** with six questions and a separate instructor solution section in the PowerPoint. All mathematical expressions in the instructional material are presented in LaTeX-style notation.

Week 13, Lecture 25

Explain in more detail what's going on here and what good answers and/or examples are. Please use LaTeX for all math expressions

Week 13, Lecture 25, LaTeX math

Please convert all math expressions to LaTeX. Don't change anything else.

Gemini Flash-Lite:

Yes. This lecture is really making one conceptual transition:

$$\text{DSP computation} \rightarrow \text{differentiable DSP computation}$$

The important message is that tensors are not “neural-network objects.” They are familiar arrays plus metadata and, optionally, a record of how they were computed. The deck states this explicitly: a tensor can represent the same waveform, spectrum, filterbank, or feature matrix students already know from DSP. Below is how I would explain the lecture in class, including what strong answers and useful examples look like.

Slides 2–4: What actually changes when a DSP array becomes a tensor?

A discrete-time signal

$$x[n]$$

is usually introduced mathematically as a sequence. If we observe L samples, we might write

$$\mathbf{x} = [x[0] \quad x[1] \quad \cdots \quad x[L-1]]^T.$$

In NumPy, that becomes an array with shape

$$(L,).$$

In PyTorch, the numerical values may be identical and the shape can still be

$$(L,).$$

but the tensor also carries several practical properties:

shape, dtype, device, requires_grad

which the deck highlights. A useful way to phrase this is:

A mathematical sequence tells us the values. A PyTorch tensor also tells the software how those values are stored and whether their computational history should be tracked.

Good answer to the slide question The slide asks:

“What extra information does the tensor carry that the mathematical sequence $x[n]$ does not?” A good student answer is:

“It carries the tensor shape, numerical data type, the device where the data live, and whether PyTorch should track operations involving the tensor for gradient computation.”

An even better answer is:

“Those properties affect both correctness and performance. Shape determines axis meaning, dtype affects precision, device determines where computation happens, and requires_grad determines whether the tensor participates in automatic differentiation.”

Slide 5: Spectrograms are already tensors

Students sometimes hear “tensor” and imagine something exotic. But a spectrogram is already naturally a rank-2 object. If the STFT is

$$X[k, m],$$

where

$$k$$

indexes frequency and

$$m$$

indexes time frames, then we can represent it as

$$\mathbf{X} \in \mathbb{C}^{F \times T}.$$

For example, with

$$N_{\text{FFT}} = 256,$$

a one-sided real-input STFT has

$$\frac{256}{2} + 1 = 129$$

frequency bins. So a possible tensor shape is

$$(129, 32).$$

The deck emphasizes that PyTorch tensors can also have complex dtype, so STFT-domain quantities can remain inside a differentiable graph. That is an important point. You do not necessarily have to reduce everything to real magnitude data before using PyTorch. Rank versus shape This is worth explicitly distinguishing. A tensor with shape

$$(129, 32)$$

has rank 2 because it has two axes. A tensor with shape

$$(32, 2, 16000)$$

has rank 3. The numbers in the shape describe axis lengths. The number of entries in the shape describes the rank. Students often confuse these. A good check is:

“What is the rank of a stereo batch with shape (32,2,16000)?” Correct answer:

$$\boxed{3}$$

not 16000, and not 32.

Slide 6: Tensor anatomy is really engineering bookkeeping

The deck says many PyTorch problems are not “machine-learning problems” at all—they are shape, dtype, or device problems. That is absolutely worth stressing to ECE students. I would compare tensor shape to units in circuit analysis. If a student writes

$$5 V + 3 A,$$

the arithmetic operation is defined syntactically, but the physical meaning is wrong. Similarly, a tensor operation might run numerically while the axes are semantically wrong. For example, suppose

$\mathbf{x}$

has shape

$$(N, C, L).$$

Then

$$x[7,1,250]$$

means:

batch item 7,
channel 1,
sample 250.

If you accidentally interpret the shape as

$$(N, L, C),$$

the values may still exist, but the computation is conceptually wrong. That is why “print the shape” is not beginner trivia. It is a genuine model-design diagnostic.

Slide 7: NumPy versus PyTorch

The deck makes a subtle but useful point: the main distinction is not really

array versus tensor

but more like

numerical array versus numerical array participating in differentiable computation.

Both NumPy and PyTorch can perform vectorized operations. For example:

$$y = x^{**2} + 3*x$$

looks nearly the same in both libraries. But if

`x.requires_grad = True`

then PyTorch can remember enough about the operation

$$y = x^2 + 3x$$

to later compute

$$\frac{dy}{dx} = 2x + 3.$$

That is the key additional capability. `torch.from_numpy` versus `torch.tensor` The deck briefly mentions a useful subtlety. If

```
t = torch.from_numpy(a)
```

then on CPU, *t* may share memory with the NumPy array *a*. So changing one can affect the other. For example:

```
a = np.array([1., 2., 3.], dtype=np.float32)
t = torch.from_numpy(a)
```

```
a[0] = 99
```

can cause

```
t[0]
```

to also become 99. By contrast,

```
t = torch.tensor(a)
```

normally creates a copy. This is a good example of why tensor behavior is partly about storage semantics, not just equations.

Slide 8: GPU acceleration is not automatically faster

The deck correctly says that GPU acceleration is a placement decision, not magic. Suppose you have a vector of only

$$L = 100$$

samples and perform one multiplication. Moving those 100 values to a GPU may cost more time than the computation itself. The benefit comes when the operation has lots of parallel work, such as

large matrix multiplication,

large batched convolution,

or

many spectrograms processed together.

A good answer to

“Why might a GPU be slower for a tiny operation?” is:

“Because transferring data and launching GPU kernels has overhead. The arithmetic must be large enough to amortize that cost.”

Slides 9–11: Shape conventions

The concept check asks students to map familiar audio objects to shapes. These are worth explaining carefully.

One mono clip Suppose one clip has

$$L = 16000$$

samples. A simple representation is

$$(16000,)$$

which is rank 1. Batch of stereo clips Suppose we have

$$N = 32$$

clips,

$$C = 2$$

channels, and

$$L = 16000$$

samples per clip. Then the common Conv1d convention is

$$(32, 2, 16000).$$

The deck later formalizes this as

$$(N, C, L),$$

where N is batch, C is channel, and L is length. Why include a channel dimension for mono? Suppose you have 32 mono clips. You might initially store them as

$$(32, 16000).$$

But a 1-D convolution layer usually expects

$$(N, C, L).$$

So for mono,

$$C = 1,$$

and you insert the singleton channel dimension:

`(32,1,16000)`

using

```
x = x.unsqueeze(1)
```

The 1 is not meaningless. It explicitly says:

There is one physical signal channel.

Slide 10: unsqueeze, permute, and reshape

These operations are mathematically simple but semantically important.

unsqueeze If

x

has shape

$(N, L),$

then

```
x = x.unsqueeze(1)
```

produces

$(N, 1, L).$

It inserts a new axis of length 1. No samples are added.

permute Suppose

A

has shape

$(8, 20, 30).$

Then

```
B = A.permute(0, 2, 1)
```

produces shape

$(8, 30, 20).$

This changes which semantic quantity belongs to each axis. That is different from simply regrouping the same entries.

reshape

This is where students can get into trouble. Suppose

$$A \in \mathbb{R}^{8 \times 20 \times 30}.$$

The total number of elements is

$$8 \cdot 20 \cdot 30 = 4800.$$

You could reshape to

$$(8, 600),$$

because

$$8 \cdot 600 = 4800.$$

But whether that is meaningful depends on what the original axes represented. A good teaching statement is:

A shape-compatible reshape is not necessarily a physically meaningful reshape.

Slide 12: Broadcasting

The deck uses a very practical DSP example: one gain per channel. Suppose

$$\text{audio.shape} = (N, C, L).$$

For stereo,

$$C = 2.$$

Suppose we want gains

$$g_0 = 0.5, \quad g_1 = 1.25.$$

A natural gain tensor is

$$\mathbf{g} = [0.5 \quad 1.25].$$

But shape

$$(2,)$$

does not say which axis should correspond to channel. We want

$$\boxed{(1, C, 1)}$$

because: 1 in the batch dimension means “reuse this gain for every batch item,” C matches the channel dimension, and the final 1 means “reuse this gain for every time sample.” So

$$\mathbf{y} = \mathbf{x} \odot \mathbf{g}$$

broadcasts correctly.

Slide 14: Why shape (2,) fails

The deck's concept check asks about

`audio.shape = (8,2,1000)`

and

`gain.shape = (2, ).`

PyTorch broadcasting aligns shapes from the rightmost dimension. So conceptually it tries to compare

`(8,2,1000)`

with

`( 2).`

The last dimensions are

`1000`

and

`2,`

which do not match and neither is 1. Therefore the operation fails. The correct gain shape is

`(1,2,1).`

Then the alignment is

`(8,2,1000)`

with

`(1,2,1),`

and every dimension is either equal or 1.

Slide 13: Matrix multiplication as DSP

The filterbank example is especially useful because it shows that PyTorch operations are just familiar linear algebra. Suppose the power spectrogram is

$$\mathbf{P} \in \mathbb{R}^{257 \times 100}.$$

Interpretation: 257 frequency bins, 100 time frames. Suppose the filterbank matrix is

$$\mathbf{W} \in \mathbb{R}^{40 \times 257}.$$

Then

$$\boxed{\mathbf{E} = \mathbf{W}\mathbf{P}}$$

has shape

$$(40, 100).$$

Why?

$$(40 \times 257)(257 \times 100) \rightarrow (40 \times 100).$$

Each output entry is

$$E_{m,t} = \sum_{k=0}^{256} W_{m,k} P_{k,t}.$$

That is simply the energy in filterbank band m at frame t . A strong answer to “What does matrix multiplication mean physically here?” is:

“Each row of $\mathbf{W}$ defines one spectral weighting function. Multiplying $\mathbf{W}$ by each spectrogram column combines the original frequency-bin powers into band energies.”

Slide 15: The conceptual turning point

The lecture then asks:

What if the DSP coefficients are unknown? That is where traditional DSP computation becomes a learnable model. Suppose we have a system

$$\hat{y} = f(x; \theta),$$

where

$$\theta$$

contains unknown parameters. We compare the predicted output

$$\hat{y}$$

against a desired output

$$y$$

using a loss

$$L(\theta).$$

Now we ask:

How should θ change so that L decreases?

The mathematical answer is the gradient:

$$\nabla_{\theta} L$$

The direction

$$-\nabla_{\theta} L$$

is locally the direction of steepest decrease. This is the bridge from signal processing to learning.

Slide 16: Chain rule by hand

The deck gives a scalar example and evaluates the gradient at $\theta = 2$. Suppose

$$u = 3\theta + 1$$

and

$$L = u^2.$$

Then

$$\frac{dL}{d\theta} = \frac{dL}{du} \frac{du}{d\theta}.$$

We have

$$\frac{dL}{du} = 2u$$

and

$$\frac{du}{d\theta} = 3.$$

Therefore,

$$\frac{dL}{d\theta} = 6u.$$

Since

$$u = 3\theta + 1,$$

we get

$$\boxed{\frac{dL}{d\theta} = 6(3\theta + 1)}$$

or

$$\boxed{\frac{dL}{d\theta} = 18\theta + 6.}$$

At

$$\theta = 2,$$

$$u = 7$$

and

$$\frac{dL}{d\theta} = 6(7) = \boxed{42}.$$

This is exactly what PyTorch later produces. What does the gradient value 42 mean? Students often treat gradients as symbolic objects. I would ask:

“What does 42 actually tell us?” A good answer:

“Near $\theta = 2$, increasing θ slightly increases the loss at approximately 42 units of loss per unit change in θ .” More formally,

$$L(\theta + \Delta\theta) \approx L(\theta) + \frac{dL}{d\theta} \Delta\theta.$$

So if

$$\Delta\theta = 0.01,$$

then approximately

$$\Delta L \approx 42(0.01) = 0.42.$$

That makes the gradient physically interpretable as local sensitivity.

Slide 17: The computational graph

The graph is

$$\theta \rightarrow u = 3\theta + 1 \rightarrow L = u^2.$$

The deck explains that the forward pass builds a directed acyclic graph and the backward pass multiplies local derivatives using the chain rule. A useful way to visualize the reverse pass is: Start with

$$\frac{dL}{dL} = 1.$$

At the $L = u^2$ node:

$$\frac{dL}{du} = 2u.$$

At the $u = 3\theta + 1$ node:

$$\frac{du}{d\theta} = 3.$$

Multiply:

$$\frac{dL}{d\theta} = \frac{dL}{du} \frac{du}{d\theta} = 2u \cdot 3.$$

Autograd is not inventing a new calculus rule. It is automating repeated application of the chain rule. Why is the graph a DAG? DAG means directed acyclic graph. Directed because computation flows from inputs to outputs. Acyclic because, within one forward evaluation, a quantity cannot depend on itself through a closed loop. For example,

$$\theta \rightarrow u \rightarrow L$$

contains no cycle. This matters because the backward traversal can process dependencies in reverse topological order.

Slide 18: Why reverse-mode autodiff is ideal for ML

The deck states the critical reason:

Many parameters, one scalar loss. Suppose

$$\theta = [\theta_1 \quad \theta_2 \quad \cdots \quad \theta_P]^T$$

with

$$P = 10^6.$$

The loss is scalar:

$$L \in \mathbb{R}.$$

We want

$$\nabla_{\theta} L = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} \\ \frac{\partial L}{\partial \theta_2} \\ \vdots \\ \frac{\partial L}{\partial \theta_p} \end{bmatrix}.$$

Reverse mode computes all of these efficiently in one backward traversal. That is exactly the problem structure encountered in neural networks and learnable DSP systems.

Slide 19: A vectorized linear model

The deck uses

$$\hat{y} = wx + b$$

with mean-square loss. Suppose the target is

$$y_i = 2.5x_i - 0.4.$$

The model begins with

$$w = 0.2, \quad b = 0.$$

Predictions are

$$\hat{y}_i = wx_i + b.$$

The loss is

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

PyTorch computes both

$$\frac{\partial L}{\partial w}$$

and

$$\frac{\partial L}{\partial b}$$

from the same backward pass. A useful hand derivation is

$$\frac{\partial L}{\partial w} = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i) x_i$$

and

$$\frac{\partial L}{\partial b} = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i).$$

This starts looking very similar to the FIR gradient later.

Slide 20: Understanding the minimal autograd API

The deck's minimal example is:

```
theta = torch.tensor(2.0, requires_grad=True)
u = 3*theta + 1
loss = u**2
loss.backward()
print(theta.grad)
```

which produces

42.

The conceptual role of each line is: `requires_grad=True` means
track operations downstream from θ .

`loss.backward()` means

$$\text{compute } \frac{\partial L}{\partial (\cdot)}$$

for all relevant upstream leaf tensors. `theta.grad` stores

$$\frac{\partial L}{\partial \theta}.$$

Why does `.grad` usually live on leaf tensors? A leaf tensor is typically a parameter created directly by the programmer, such as

```
theta = torch.tensor(..., requires_grad=True)
```

The intermediate value

$$u = 3\theta + 1$$

is temporary. Autograd needs u 's information for the backward pass, but storing a persistent `.grad` tensor for every intermediate would consume unnecessary memory. A good answer to the slide question is:

“We normally care about gradients of the parameters we intend to update. Intermediate gradients are used internally during backpropagation but are not all retained as user-visible `.grad` fields by default.”

Slide 21: Gradient accumulation

This is one of the most common PyTorch surprises. The deck demonstrates:

$$L = x^2$$

at

$$x = 3.$$

Then

$$\frac{dL}{dx} = 2x = 6.$$

After the first `backward()`,

$$x.grad = 6.$$

After a second `backward()` without clearing,

$$x.grad = 12.$$

The reason is:

PyTorch accumulates gradients

rather than replacing them. This behavior is mathematically useful. If a parameter influences the loss through several paths,

$$L = L_1 + L_2,$$

then

$$\frac{\partial L}{\partial \theta} = \frac{\partial L_1}{\partial \theta} + \frac{\partial L_2}{\partial \theta}.$$

So summing gradient contributions is exactly what the chain rule requires. The danger is accidentally accumulating across training iterations. Typical training loop The deck gives the standard pattern:

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Conceptually:

clear old gradient
compute current gradient
update parameters.

If you skip the clearing step, the parameter update can use

$$g_{\text{old}} + g_{\text{new}}$$

instead of just

$$g_{\text{new}}.$$

Slide 22: `no_grad()` versus `detach()`

These are related but different.

`torch.no_grad()` This is a context manager. Operations executed inside it do not create autograd history. For example:

```
with torch.no_grad():
    y = model(x)
```

This is useful during inference because we do not need gradients. It saves memory and computation.

`.detach()` Suppose

y

already belongs to a graph. Then

```
z = y.detach()
```

creates a tensor with the same numerical values but disconnected from that graph. So gradients through z do not flow backward to the operations that created y . A good use is:

```
plot_value = y_hat.detach().cpu().numpy()
```

because plotting does not need gradient history.

Good answer: What is the difference?

“no_grad() prevents graph construction for operations executed inside a region. detach() takes an existing tensor and creates a view of its values that no longer participates in the current gradient graph.” That is a very solid answer.

Slide 23: FIR filtering as a learnable model

This is the strongest DSP connection in the lecture. A causal FIR filter is

$$\hat{y}[n] = \sum_{k=0}^{M-1} h[k]x[n-k]$$

where the tap coefficients

$$h[k]$$

are now treated as unknown parameters. Suppose we know a target output

$$y[n].$$

Define the error

$$e[n] = \hat{y}[n] - y[n].$$

Use mean-square error:

$$L = \frac{1}{N} \sum_n e^2[n]$$

The deck points out that the resulting gradient is correlation-like.

Let's derive it.

Deriving the FIR gradient

We want

$$\frac{\partial L}{\partial h[r]}.$$

Start with

$$L = \frac{1}{N} \sum_n (\hat{y}[n] - y[n])^2.$$

Then

$$\frac{\partial L}{\partial h[r]} = \frac{1}{N} \sum_n 2 (\hat{y}[n] - y[n]) \frac{\partial \hat{y}[n]}{\partial h[r]}.$$

Since

$$\hat{y}[n] = \sum_k h[k] x[n - k],$$

we have

$$\frac{\partial \hat{y}[n]}{\partial h[r]} = x[n - r].$$

Therefore,

$$\boxed{\frac{\partial L}{\partial h[r]} = \frac{2}{N} \sum_n e[n] x[n - r]}$$

This is a beautiful result for DSP students. It says:

The gradient for tap r is obtained by correlating the current output error with the input delayed by r samples. That should feel very familiar to anyone who has seen adaptive filtering.

Connection to LMS adaptive filtering

This is a particularly valuable extension. The LMS update has the general form

$$h_r[m + 1] = h_r[m] + \mu e[m] x[m - r]$$

depending on sign convention. Gradient descent has the generic form

$$h_r^{(m+1)} = h_r^{(m)} - \eta \frac{\partial L}{\partial h[r]}.$$

These have essentially the same structure:

$$\boxed{\text{coefficient update} \propto \text{error} \times \text{input}}$$

So modern autograd is not disconnected from classical adaptive signal processing. It generalizes the same gradient idea to arbitrarily complicated differentiable systems. This is one of the best conceptual bridges in the entire lecture.

Important conv1d convention

The slide correctly warns that `torch.nn.functional.conv1d` implements cross-correlation in the deep-learning convention. Textbook discrete-time convolution is

$$y[n] = \sum_k h[k]x[n-k].$$

Deep-learning “convolution” often computes something closer to

$$y[n] = \sum_k w[k]x[n+k]$$

depending on indexing and padding conventions. So if you want exact textbook convolution ordering, you may need to flip the kernel.

A good student observation is:

“The operation is still linear and shift structured, but the coefficient indexing convention differs.”

Exactly.

Slide 24: System identification by gradient descent

The deck’s FIR demo has target taps

$$[0.15, 0.50, 0.25, -0.10]$$

and the learned taps converge to nearly the same values. This is a simple system-identification problem. Imagine that the actual system is

$$h_{\text{true}}[k].$$

We feed a known input

$$x[n]$$

through the system and observe

$$y[n].$$

Our model has unknown coefficients

$$h[k].$$

We calculate

$$\hat{y}[n]$$

and the loss

$$L.$$

Autograd calculates

$$\nabla_h L.$$

Then gradient descent updates

$$h^{(m+1)} = h^{(m)} - \eta \nabla_h L$$

until the modeled filter matches the target system.

The key teaching statement from the slide is excellent:

Autograd did not “understand filters.” It simply differentiated the operations you programmed. That is exactly what students need to understand before neural networks arrive.

A useful question: Is this “machine learning”?

Yes.

The model contains parameters

$$h[k]$$

that are learned from data by minimizing an objective. So even though the model is “just an FIR filter,” it satisfies the core ML pattern:

$$\text{parameterized model} + \text{data} + \text{loss} + \text{optimization}$$

This helps demystify neural networks.

Slide 25: Common failure modes

The deck emphasizes that many early PyTorch mistakes are bookkeeping mistakes. These are worth turning into diagnosis questions.

Wrong axis order

Expected:

$$(N, C, L)$$

received:

$$(N, L, C).$$

Good diagnosis:

“The numerical data are present, but the layer is interpreting time samples as channels.”

Mixed device

Input on CPU:

$x \in \text{CPU memory}$

parameters on GPU:

$\theta \in \text{GPU memory.}$

The operation cannot proceed because the operands are not in compatible memory spaces.

Gradient accumulation

Symptom: Training becomes unstable or parameter updates become unexpectedly large.

Likely cause: Forgot

`optimizer.zero_grad()`

Detached graph

Symptom: A parameter's gradient is None even though you expect it to affect the loss.

Likely cause: An intermediate tensor was detached or created under `no_grad()`.

In-place operation An in-place modification can overwrite a value autograd needs for the backward pass. While learning, functional-style code is safer:

`y = x + 1`

rather than aggressively modifying tensors in place.

Homework 1: Tensor shapes

The deck asks students to assign sensible shapes. Good answers are:

One mono clip, 16000 samples

`(16000,)`

rank:

`1`

32 stereo clips

`(32,2,16000)`

rank:

`3`

One STFT

`(257,101)`

possibly complex. Rank:

`2`

Batch of 32 one-channel spectrograms

`(32,1,257,101)`

rank:

`4`

The important thing is not just the numbers. Students should state what each axis means.

Homework 2: Broadcasting gains

Given

`x.shape = (24,2,8000),`

we want

$g_L = 0.5$

and

$g_R = 1.2.$

Use

`g.shape = (1,2,1)`

with

```
g = torch.tensor([0.5, 1.2]).view(1,2,1)
y = x * g
```

The result has shape

`(24,2,8000)`.

Homework 3: Filterbank multiplication

Given

$$\mathbf{P} \in \mathbb{R}^{257 \times 100}$$

and

$$\mathbf{W} \in \mathbb{R}^{40 \times 257},$$

compute

$$\boxed{\mathbf{E} = \mathbf{W}\mathbf{P}}$$

with

$$\mathbf{E} \in \mathbb{R}^{40 \times 100}.$$

Interpretation: 40 filterbank bands by 100 time frames. The solution in the appendix gives exactly this interpretation.

Homework 4: Chain rule

The problem uses

$$u = 2\theta - 3$$

and

$$L = (u + 1)^2.$$

First simplify:

$$u + 1 = 2\theta - 2.$$

Then

$$L = (2\theta - 2)^2.$$

By the chain rule,

$$\frac{dL}{d\theta} = 2(2\theta - 2) \cdot 2.$$

Therefore,

$$\boxed{\frac{dL}{d\theta} = 8\theta - 8}$$

At

$$\theta = 4,$$

$$\frac{dL}{d\theta} = 32 - 8 = \boxed{24}.$$

The instructor appendix confirms this result.

Homework 5: Accumulation

For

$$L = x^2$$

at

$$x = 3,$$

the derivative is

$$\frac{dL}{dx} = 2x = 6.$$

First backward:

$$\boxed{6}$$

Second backward without clearing:

$$\boxed{12}$$

because PyTorch accumulates gradients. The solution appendix states exactly this.

Homework 6: FIR gradient interpretation

The key result is

$$\boxed{\frac{\partial L}{\partial h[r]} = \frac{2}{N} \sum_n e[n] x[n-r].}$$

The strongest interpretation is:

“The gradient for each tap is proportional to the correlation between the current output error and the input sequence delayed to that tap.” That is the DSP meaning of backpropagation through an FIR filter. The deck’s solution appendix makes the same point.

Weekly quiz: Good answers

The quiz asks several very good conceptual questions.

Q1

16 mono clips, each 4000 samples, using (N, C, L) :

$(16, 1, 4000)$

Q2

For input shape

$(8, 2, 1000)$,

one independent gain per channel should have shape

$(1, 2, 1)$.

Q3

Every PyTorch tensor automatically tracks gradients. Answer:

False

A floating-point or complex tensor typically tracks gradients only when `requires_grad=True` or when it is downstream from such a tensor.

Q5

Why is reverse-mode autodiff appropriate for modern ML? Good answer:

“Because one scalar loss depends on many parameters. Reverse mode computes all components of $\nabla_{\theta} L$ efficiently in a single backward traversal.” That matches the instructor solution.

Q6

What happens if gradients are never cleared? Good answer:

“They accumulate across iterations, so later updates use the sum of old and new gradients rather than just the current gradient.” Again, exactly the intended concept.

The most important conceptual connection to DSP

I would make this connection very explicit near the end of the lecture. Traditional DSP often looks like

$$x[n] \rightarrow H \rightarrow y[n]$$

where the system H is known. Modern ML often looks like

$$x \rightarrow f(x; \theta) \rightarrow \hat{y}$$

where the system contains unknown parameters

$$\theta.$$

Then we define

$$L(\hat{y}, y)$$

and compute

$$\nabla_{\theta} L.$$

So the fundamental change is not:

“We stopped doing signal processing and started doing machine learning.” It is:

We made some signal-processing operations parameterized and differentiable.

That is a much more coherent transition for ECE students.

What I would expect a senior student to know By the end of the lecture, a strong senior should be able to explain: (N, C, L) and why axis order matters; how broadcasting works; why a filterbank can be written as

$$\mathbf{E} = \mathbf{W}\mathbf{P};$$

what

`requires_grad=True` does; why

`loss.backward()` computes gradients; and why gradients need to be cleared between training iterations.

What I would expect a first-year graduate student to know In addition, I would want the graduate student to be able to derive:

$$\frac{dL}{d\theta} = \frac{dL}{du} \frac{du}{d\theta'}$$

explain why reverse-mode autodiff is efficient for

$$L: \mathbb{R}^P \rightarrow \mathbb{R},$$

derive

$$\frac{\partial L}{\partial h[r]} = \frac{2}{N} \sum_n e[n] x[n-r],$$

and recognize that this FIR gradient has the same error-times-input structure that appears in classical adaptive filtering. One sentence students should leave with The best answer to the minute-paper question at the end of the deck would be something like:

“A tensor that merely contains a signal stores numerical samples and metadata, while a tensor participating in a computational graph also retains the differentiable relationships needed to compute how a final loss changes with respect to upstream parameters.”

That captures the entire lecture. The deck itself summarizes the same progression: tensors store familiar DSP data with shape/dtype/device metadata, tensor operations express vectorized DSP, autograd constructs a dynamic graph, and unknown DSP parameters can then be optimized from

$$\nabla_{\theta} L.$$

Week 13, Lecture 26

Prompt: Can you create slides and python code for this 75 minute class of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schaffer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well. Please write math expressions in LaTeX.

Chat GPT (Medium):

Created. I kept Lecture 26 as a direct continuation of Lecture 25, with engineered DSP features feeding a small, inspectable MLP rather than jumping immediately to raw-audio deep learning.

Files

- [Complete Lecture 26 instructor package](#)
- [PowerPoint deck — 41 slides](#)
- [Microsoft PowerPoint-compatible version](#)
- [Jupyter notebook](#)
- [Executed and verified notebook](#)
- [Standalone Python code](#)
- [Homework, quiz, and complete solutions](#)
- [Teaching-flow README](#)

The Microsoft-compatible deck was reopened and re-saved through Office Open XML and rendered successfully afterward. The executed notebook contains **no execution errors**.

The 75-minute lecture is organized as follows:

Architecture, about 20 minutes. Students begin with an eight-element DSP feature vector containing spectral centroid, ZCR, spectral roll-off, RMS energy, spectral flux, and three MFCCs. The basic layer is developed from

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

with explicit shape reasoning for single examples and minibatches. The deck then shows algebraically why multiple linear layers without an activation still collapse to one affine map:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2).$$

That motivates ReLU,

$$\text{ReLU}(z) = \max(0, z),$$

as the ingredient that allows genuinely nonlinear decision regions.

Loss and optimization, about 25 minutes. The output layer produces three raw logits. Softmax is introduced as

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}},$$

and multiclass cross-entropy as

$$L = -\log p_y$$

or, over a minibatch,

$$L = -\frac{1}{N} \sum_{i=1}^N \log p_{i, y_i}.$$

A numerical example using

$$\mathbf{z} = [2, 1, 0]$$

gives

$$p_0 \approx 0.665$$

and

$$L \approx 0.408.$$

The deck explicitly emphasizes that `nn.CrossEntropyLoss()` expects **raw logits**, so students should not apply softmax in the network before the loss.

Gradient descent is tied back to Lecture 25 through

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L,$$

and Adam is introduced through its first- and second-moment estimates so students understand that `optimizer.step()` is doing more than simply multiplying by a fixed learning rate.

PyTorch training loop, about 30 minutes. Each operation gets its own slide before the complete loop is shown:

```
logits = model(xb)
loss = criterion(logits, yb)

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

The conceptual interpretation is:

predict → measure error → clear stale gradients → compute current gradients → update parameters

The slides also distinguish

`loss.backward()`

from

`optimizer.step()`,

which is a distinction students frequently blur: the first **computes gradients**, while the second **changes parameters**.

The code uses the same synthetic Kick/Snare/Cymbal DSP feature dataset from Week 12. The MLP is

$$8 \rightarrow 16 \rightarrow 8 \rightarrow 3$$

with ReLU hidden activations, cross-entropy loss, and Adam. The notebook includes training-loss curves, train/test accuracy curves, logits versus probabilities, confusion-matrix evaluation, and gradient-shape inspection.

The synthetic dataset is intentionally clean, so the trained MLP can reach essentially perfect held-out classification. I call that out explicitly in the slides rather than presenting it as representative of real audio; the deck contrasts it with the overlap, microphone effects, room effects, performer variability, and session leakage that students should expect in actual signal-classification problems.

The package also contains **7 homework problems** with worked solutions and a **6-question weekly quiz** with solutions. The homework covers layer shapes, why nonlinear activations are necessary, ReLU, hand calculation of cross-entropy, a gradient-descent update, training-loop reasoning, and why scaling heterogeneous DSP features matters. All mathematical expressions in the instructional material are rendered in LaTeX-style notation.

Week 13, Lecture 26, LaTeX math

Explain in more detail what's going on here and what good answers and/or examples are. Please use LaTeX for all math expressions

Chat GPT:

Please convert all math expressions to LaTeX. Do not change anything else.

Gemini Flash Lite:

Absolutely. Lecture 26 is the point where the pieces from Lecture 25 become a complete learning system. The core conceptual pipeline is

DSP features $\rightarrow$ MLP $\rightarrow$ logits $\rightarrow$ cross-entropy loss $\rightarrow \nabla_{\theta} L \rightarrow$ Adam update

The deck explicitly frames the lecture this way: Lecture 25 gave students tensors and gradients; Lecture 26 asks what computation should be differentiated, and how does that computation become a classifier?

Slides 2–3: From a signal to an MLP input

The important first step is that this lecture is not yet classifying raw audio samples. Instead, the DSP front end has already converted each recording into engineered features such as:

spectral centroid,
zero-crossing rate,
spectral roll-off,
RMS energy,
spectral flux,

and several MFCCs. The deck uses eight features total, so one audio clip becomes a vector

$\mathbf{x} \in \mathbb{R}^8$

and a minibatch of N clips becomes

$$\mathbf{X} \in \mathbb{R}^{N \times 8}.$$

The three output classes are Cymbal, Kick, and Snare, so the network eventually produces three values per example. This is a good point to make the DSP/ML division explicit:

DSP determines what information we present to the classifier. The MLP learns how to combine that information to separate the classes. For example, an engineer might already suspect that a cymbal tends to have:

high spectral centroid,

high zero-crossing rate,

and broadband spectral energy. But instead of manually writing the classification rule, the MLP learns combinations of these quantities.

Slide 4: What is one MLP layer actually doing?

The fundamental layer is

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

followed by an activation

$$\mathbf{h} = f(\mathbf{z}).$$

For one hidden neuron,

$$z_j = \sum_{i=1}^D W_{ji} x_i + b_j.$$

So every neuron takes a weighted combination of the input features. Suppose

$$\mathbf{x} = \begin{bmatrix} \text{centroid} \\ \text{ZCR} \\ \text{rolloff} \\ \text{RMS} \\ \text{flux} \\ \text{MFCC}_1 \\ \text{MFCC}_2 \\ \text{MFCC}_3 \end{bmatrix}.$$

One hidden unit might effectively learn something resembling

$$z_1 = 0.8(\text{centroid}) + 0.5(\text{ZCR}) - 0.2(\text{RMS}) + 0.7(\text{flux}) + \dots + b_1.$$

That neuron is therefore detecting a learned combination of DSP characteristics rather than one individual feature.

The deck describes W as mixing the input features, b as shifting the affine boundary, and $f(\cdot)$ as providing the nonlinear behavior required for complex decision regions.

What does the bias do?

Students often understand weights better than bias. Consider the one-dimensional case:

$$z = wx + b.$$

The decision point where

$$z = 0$$

is

$$wx + b = 0,$$

so

$$x = -\frac{b}{w}.$$

Without bias,

$$b = 0,$$

and the boundary is forced through the origin. With bias, the boundary can move. In multiple dimensions, the equation

$$\mathbf{w}^T \mathbf{x} + b = 0$$

defines a hyperplane. The vector

$$\mathbf{w}$$

controls its orientation, while

$$b$$

controls its offset. A good student answer to

“Why do we need a bias?” is:

“Without the bias, every affine boundary would be constrained to pass through the origin. The bias lets the network shift the threshold independently of the feature origin.”

Slide 5: Shape reasoning

The first layer is

Linear(8,16).

For one input,

$$\mathbf{x} \in \mathbb{R}^8.$$

The weight matrix is

$$\mathbf{W} \in \mathbb{R}^{16 \times 8}$$

and the bias is

$$\mathbf{b} \in \mathbb{R}^{16}.$$

Thus,

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \in \mathbb{R}^{16}.$$

For a minibatch,

$$\mathbf{X} \in \mathbb{R}^{N \times 8},$$

PyTorch effectively computes

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T + \mathbf{b}$$

so

$$\mathbf{Z} \in \mathbb{R}^{N \times 16}.$$

The deck explicitly uses the shapes

$$(N, 8) \rightarrow (N, 16)$$

and notes that Linear(8,16) contains

$$16 \cdot 8 + 16 = \boxed{144}$$

trainable parameters.

A good question is:

“Why is the PyTorch weight shape (16,8), not (8,16)?” Good answer: “PyTorch stores one row of weights per output neuron. Each of the 16 neurons needs 8 input weights.”

Slide 6: ReLU The activation is

$$\text{ReLU}(z) = \max(0, z).$$

For

$$\mathbf{z} = \begin{bmatrix} -2 \\ -0.5 \\ 0 \\ 1.2 \\ 4 \end{bmatrix},$$

we obtain

$$\text{ReLU}(\mathbf{z}) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1.2 \\ 4 \end{bmatrix}.$$

The derivative is

$$\frac{d}{dz} \text{ReLU}(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

At

$$z = 0,$$

the mathematical derivative is not unique, so implementations use a subgradient convention. The deck correctly emphasizes ReLU's cheap, piecewise-linear nature and its ability to create different affine behavior in different feature-space regions. Why is ReLU so important? This is probably the most important architecture question in the lecture. Suppose we stack two affine layers but do not use a nonlinear activation. First layer:

$$\mathbf{h} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1.$$

Second layer: $\mathbf{y} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2$. Substitute:

$$\mathbf{y} = \mathbf{W}_2 (\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2.$$

Expanding,

$$\mathbf{y} = \mathbf{W}_2 \mathbf{W}_1 \mathbf{x} + \mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2.$$

Define

$$\mathbf{W}_{\text{eq}} = \mathbf{W}_2 \mathbf{W}_1$$

and

$$\mathbf{b}_{\text{eq}} = \mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2.$$

Then

$$\boxed{\mathbf{y} = \mathbf{W}_{\text{eq}} \mathbf{x} + \mathbf{b}_{\text{eq}}.}$$

So two affine layers have collapsed into one affine layer. The same argument applies to 100 affine layers.

The deck makes exactly this point and gives the strong answer: “The composition of affine maps is affine; ReLU breaks that collapse.”

What is a really good answer to “Why do we need ReLU?”

Weak answer:

“Because neural networks need activation functions.”

Better answer:

“Without nonlinear activations, multiple layers collapse into one affine transformation.”

Excellent answer:

“ReLU creates different affine regimes depending on which hidden units are active. The resulting network is piecewise affine, so the overall class boundaries can be nonlinear even though each local region is linear.”

That is the deeper idea. Piecewise-linear interpretation

Suppose one ReLU unit computes

$$h = \max(0, \mathbf{w}^T \mathbf{x} + b).$$

The feature space is divided into two regions:

$$\mathbf{w}^T \mathbf{x} + b \leq 0$$

and

$$\mathbf{w}^T \mathbf{x} + b > 0.$$

In the first region,

$$h = 0.$$

In the second,

$$h = \mathbf{w}^T \mathbf{x} + b.$$

With many ReLU units, different combinations of units switch on and off in different areas. So the network becomes a collection of locally affine functions stitched together. That is why the slide's phrase

“piecewise-linear decision surface” is much better than thinking of neurons as biological metaphors.

Slide 8: The actual network

The deck uses

$$\boxed{8 \rightarrow 16 \rightarrow 8 \rightarrow 3}$$

with ReLU after both hidden layers. So:

$$\mathbf{x} \in \mathbb{R}^8$$

becomes

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1),$$

with

$$\mathbf{h}_1 \in \mathbb{R}^{16}.$$

Then

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2),$$

with

$$\mathbf{h}_2 \in \mathbb{R}^8.$$

Finally,

$$\mathbf{z} = \mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3,$$

with

$$\boxed{\mathbf{z} \in \mathbb{R}^3.}$$

Those final three numbers are the logits. Parameter count

Layer 1:

$$8 \cdot 16 + 16 = 144$$

Layer 2:

$$16 \cdot 8 + 8 = 136$$

Layer 3:

$$8 \cdot 3 + 3 = 27$$

Total:

$$144 + 136 + 27 = \boxed{307}.$$

This is useful to emphasize because the model is small enough that students can actually understand every parameter group.

Slides 9–10: What does the PyTorch module mean?

The code is essentially:

```
self.net = nn.Sequential(  
    nn.Linear(8, 16),  
    nn.ReLU(),  
    nn.Linear(16, 8),  
    nn.ReLU(),  
    nn.Linear(8, 3),  
)
```

The input has shape

$$(N, 8)$$

and the output has shape

$$(N, 3).$$

The deck emphasizes no softmax in the network because CrossEntropyLoss expects raw logits. A strong shape answer for a minibatch of 64 examples is:

$$\boxed{\mathbf{X}: (64, 8)}$$

first weight matrix:

$$\boxed{\mathbf{W}_1: (16, 8)}$$

and hidden output:

$$\boxed{\mathbf{H}_1: (64, 16)}.$$

That matches the concept check in the deck.

Slide 11: What exactly is a logit?

Suppose the network returns

$$\mathbf{z} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}.$$

These are not probabilities. They do not need to satisfy

$$z_k \geq 0$$

and do not need to sum to

$$1.$$

A logit is simply an unconstrained score. The predicted class can already be found from

$$\hat{y} = \arg \max_k z_k.$$

Since softmax preserves ordering,

$$\arg \max_k z_k = \arg \max_k p_k.$$

The deck stresses this distinction and the rule to feed raw logits into `nn.CrossEntropyLoss()`.

Softmax

To interpret logits as probabilities,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

For

$$\mathbf{z} = [2, 1, 0],$$

we calculate

$$e^2 \approx 7.389,$$

$$e^1 \approx 2.718,$$

and

$$e^0 = 1.$$

The denominator is

$$7.389 + 2.718 + 1 = 11.107.$$

Therefore,

$$p_0 = \frac{7.389}{11.107} \approx \boxed{0.665}.$$

Similarly,

$$p_1 \approx 0.245$$

and

$$p_2 \approx 0.090.$$

Notice

$$p_0 + p_1 + p_2 = 1.$$

Slides 12–13: Cross-entropy

For one sample with correct class y ,

$$\boxed{L = -\ln p_y.}$$

The deck emphasizes two limiting cases.

If

$$p_y \rightarrow 1,$$

then

$$-\ln p_y \rightarrow 0.$$

So a confident correct prediction gives very small loss.

If

$$p_y \rightarrow 0,$$

then

$$-\ln p_y \rightarrow \infty.$$

So a confident wrong prediction receives a large penalty.

Worked example

If class 0 is correct and

$$p_0 \approx 0.6652,$$

then

$$L = -\ln(0.6652) \approx \boxed{0.4076}.$$

That is the worked result in the deck.

Interpretation:

The model favors the correct class, but only moderately. The loss is below 1, but not close to zero.

Useful comparison

Suppose

$$p_y = 0.9.$$

Then

$$L = -\ln(0.9) \approx 0.105.$$

If

$$p_y = 0.5,$$

then

$$L = -\ln(0.5) \approx 0.693.$$

If

$$p_y = 0.1,$$

then

$$L = -\ln(0.1) \approx 2.303.$$

So cross-entropy penalizes increasing confidence in the wrong answer quite strongly.

Why cross-entropy rather than classification accuracy? This is an important extension.

Accuracy changes discretely.

Suppose the true class is class 0. Prediction A:

$$\mathbf{p} = [0.34, 0.33, 0.33].$$

Prediction B:

$$\mathbf{p} = [0.99, 0.005, 0.005].$$

Both are technically correct and both give accuracy

100%.

But they clearly do not represent equally good models.

Cross-entropy distinguishes them:

$$L_A = -\ln(0.34) \approx 1.079,$$

whereas

$$L_B = -\ln(0.99) \approx 0.010.$$

So cross-entropy gives a smooth learning signal.

Good answer to “Why not optimize accuracy directly?”

“Accuracy is piecewise constant with respect to the weights and therefore does not provide useful gradients. Cross-entropy is differentiable and measures the quality of the class probabilities, not just whether the top class happened to be correct.” That is an excellent graduate-level answer.

Slide 14: Why not call softmax yourself?

The deck says:

logits: $(N, 3)$

and

labels: $(N,)$

with labels such as

Cymbal $\rightarrow 0$, Kick $\rightarrow 1$, Snare $\rightarrow 2$.

CrossEntropyLoss expects integer class indices rather than one-hot targets in the standard use case. The correct pattern is:

```
logits = model(x)
loss = criterion(logits, y)
```

not:

```
prob = torch.softmax(model(x), dim=1)
loss = criterion(prob, y)
```

Why?

Because PyTorch's cross-entropy combines a numerically stable log-softmax with negative log-likelihood internally.

A very good answer is:

“Applying softmax first is redundant and can make the numerical calculation less stable. CrossEntropyLoss is designed to operate directly on logits.”

Slide 15: The beautiful cross-entropy gradient

One of the strongest mathematical points in the lecture is the derivative of softmax cross-entropy with respect to logits. For each class k ,

$$\frac{\partial L}{\partial z_k} = p_k - \mathbb{1}[k = y].$$

The deck states the interpretation: for the true class the derivative becomes $p_y - 1$, while for every wrong class it is p_k . Let's see why this is intuitive.

Suppose

$$\mathbf{p} = [0.665, 0.245, 0.090]$$

and true class is 0. Then

$$\frac{\partial L}{\partial z_0} = 0.665 - 1 = \boxed{-0.335}.$$

For class 1,

$$\frac{\partial L}{\partial z_1} = 0.245.$$

For class 2,

$$\frac{\partial L}{\partial z_2} = 0.090.$$

Gradient descent updates parameters in the negative-gradient direction.

Because the correct logit's gradient is negative, gradient descent tends to increase z_0 .

Because the incorrect logits have positive gradients, gradient descent tends to decrease z_1 and z_2 .

So cross-entropy is doing exactly what we want:

raise the correct score, lower the competing scores.

Slide 16: Gradient descent

The basic parameter update is

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L.$$

Here,

$$\theta$$

means all trainable parameters:

$$\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \mathbf{W}_3, \mathbf{b}_3.$$

The learning rate is

$$\eta.$$

The deck summarizes the consequences appropriately:

large

$$\eta$$

can produce overshooting or instability;

small

$$\eta$$

can make learning painfully slow. Simple numerical update Suppose

$$\theta_t = 1.5,$$

$$\eta = 0.02,$$

and

$$\frac{\partial L}{\partial \theta} = 4.$$

Then

$$\theta_{t+1} = 1.5 - 0.02(4).$$

Therefore,

$$\theta_{t+1} = 1.42.$$

That is also one of the homework problems in the deck.

Why negative gradient?

The gradient

$$\nabla_{\theta} L$$

points toward the local direction of greatest increase in the loss. Therefore,

$$-\nabla_{\theta} L$$

points toward the local direction of steepest decrease.

A strong student answer:

“Gradient descent uses the negative gradient because the gradient itself points uphill.”

Slide 17: What is Adam doing?

Adam is more sophisticated than

$$\theta_{t+1} = \theta_t - \eta g_t.$$

The deck introduces two running quantities.

The first moment estimate is approximately

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

where

$$g_t = \nabla_{\theta} L_t.$$

This behaves somewhat like momentum: it smooths the direction of recent gradients.

The second moment estimate is

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

This tracks gradient scale.

After bias correction, Adam uses an update approximately of the form

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Intuition for Adam

Suppose one parameter repeatedly has very large gradients while another has tiny gradients. Plain gradient descent applies the same global learning rate η . Adam adapts the effective step size according to each parameter's gradient history.

A good answer to

“Why use Adam?” is:

“Adam combines momentum-like averaging of gradients with adaptive scaling based on recent squared-gradient magnitude.”

A bad answer would be:

“Adam is always better.”

The deck correctly says Adam is a strong default, not a guarantee.

Slide 18: backward() versus step()

This distinction is absolutely fundamental. The deck separates the responsibilities clearly.

`loss.backward()` computes

$$\nabla_{\theta} L$$

and stores those derivatives in `parameter.grad`. It does not change the network parameters. `optimizer.step()` reads those gradients and updates the parameters according to Adam's rule. So:

`backward() = differentiate`

whereas

`step() = change parameters.`

This is probably the single most important answer students should give correctly.

Slides 19–25: The five-step training loop

The deck gives:

1. forward

2. loss

3. zero gradients

4. backward

5. update.

Let's unpack each.

Step 1: Forward

`logits = model(xb)`

Suppose minibatch size is B . Then

$$\mathbf{X}_b \in \mathbb{R}^{B \times 8}.$$

The network computes

$$\mathbf{H}_1 = \text{ReLU}(\mathbf{X}_b \mathbf{W}_1^T + \mathbf{b}_1),$$

then

$$\mathbf{H}_2 = \text{ReLU}(\mathbf{H}_1 \mathbf{W}_2^T + \mathbf{b}_2),$$

and finally

$$\mathbf{Z} = \mathbf{H}_2 \mathbf{W}_3^T + \mathbf{b}_3.$$

Thus,

$$\boxed{\mathbf{Z} \in \mathbb{R}^{B \times 3}.$$

During this forward pass, autograd records the operations required for the later backward pass. No parameter update has happened yet.

Step 2: Compute the loss

`loss = criterion(logits, yb)`

For a minibatch of size B ,

$$\boxed{L = -\frac{1}{B} \sum_{i=1}^B \ln p_{i,y_i}.$$

This produces one scalar. That scalar summarizes how well the entire minibatch is being classified.

The deck also makes an important distinction:

loss is a training objective

while

accuracy is an evaluation metric.

They are related, but they are not the same quantity.

Example where loss improves but accuracy does not

Suppose four samples are all already classified correctly. Initially their correct-class probabilities are

$$[0.51, 0.52, 0.53, 0.54].$$

Accuracy:

100%.

After training, probabilities become

[0.90,0.91,0.92,0.93].

Accuracy is still

100%.

But cross-entropy has decreased dramatically.

So a falling loss can represent meaningful improvement even when accuracy is unchanged.

Step 3: Clear gradients

`optimizer.zero_grad()`

PyTorch accumulates gradients. Suppose the previous minibatch left

g_{old}

in the parameter's `.grad`. Then calling `backward()` again produces

$g_{\text{old}} + g_{\text{new}}$.

Normally that is not what we want. Therefore we reset before computing the current minibatch gradient. The deck distinguishes intentional gradient accumulation from accidental accumulation.

A good answer is:

“`zero_grad()` clears stale parameter-gradient buffers so the next backward pass represents the current minibatch rather than a sum over unrelated previous minibatches.”

Could accumulation ever be intentional? Yes.

Suppose GPU memory allows batch size

$B = 32$

but you want an effective batch size of

128.

You could accumulate gradients over four minibatches before calling `optimizer.step()`.

Conceptually,

$g_{\text{effective}} = g_1 + g_2 + g_3 + g_4$.

Then perform one update.

So gradient accumulation itself is not wrong. Unintended gradient accumulation is wrong.

Step 4: Backward

```
loss.backward()
```

Now PyTorch traverses the graph backward. The path is conceptually

$L \rightarrow \text{cross-entropy} \rightarrow \text{logits} \rightarrow \text{output layer} \rightarrow \text{ReLU} \rightarrow \text{hidden layer} \rightarrow \dots$

and it computes

$$\nabla_{\theta} L.$$

Those gradients are stored in `.grad` tensors. Again:

`backward()` does not update parameters.

The deck explicitly stresses this.

Step 5: Update

```
optimizer.step()
```

Adam reads the gradients,

$$\nabla_{\theta} L,$$

updates its internal moment estimates, and changes

$$\theta_t$$

to

$$\theta_{t+1}.$$

Before `step()`:

$$\theta_t \quad \text{and} \quad \nabla_{\theta} L.$$

After `step()`:

$$\theta_{t+1}.$$

The gradients themselves remain until cleared. A new forward pass is required before predictions reflect the updated weights.

The complete loop

The full idiom is:

```
for epoch in range(num_epochs):  
    model.train()
```

```

for xb, yb in train_loader:
    logits = model(xb)
    loss = criterion(logits, yb)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

```

The deck correctly asks students to distinguish what repeats and what persists. Every minibatch gets:

new input,
new forward computation,
new loss,
new gradients.

What persists?

model parameters

and

Adam optimizer state.

What is an epoch? Suppose training contains

$$N_{\text{train}} = 1000$$

samples and batch size is

$$B = 50.$$

Then one epoch contains approximately

$$\frac{1000}{50} = 20$$

minibatch updates. So:

1 epoch = 1 pass through the training set.

If we train for 100 epochs, the model sees each training observation approximately 100 times, though batching and shuffling change the order.

Slide 26: Evaluation is not training

Evaluation code is:

```
model.eval()

with torch.no_grad():
    logits = model(X_test)
    pred = logits.argmax(dim=1)
```

The deck explains both pieces.

`model.eval()` switches any layers with different train/test behavior—such as dropout or batch normalization—into evaluation mode.

`torch.no_grad()` tells PyTorch not to record gradient history.

That saves memory and computation because we are not going to call `loss.backward()` during inference.

Validation versus test

A very important experimental-design point:

Use the training set to fit parameters. Use the validation set for decisions such as

hidden layer size,

learning rate,

number of epochs,

or architecture. Use the test set once, at the end, to estimate final performance. If you keep tuning based on test-set accuracy, then the test set is effectively becoming part of training.

Slide 27: DSP/ML hygiene

This slide may actually be more important than another neural-network equation. The deck highlights standardization, stratification, and data leakage.

Feature scaling

Your DSP features can have wildly different scales. For example,

centroid ≈ 3000 Hz,

while

ZCR ≈ 0.1 .

Without scaling, some parameters may experience much larger gradients than others. Standardization gives

$$z_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Then each standardized feature has approximately

$$\mu_j = 0$$

and

$$\sigma_j = 1.$$

This tends to improve optimization conditioning. As before:

$$\mu_j, \sigma_j$$

must be estimated from training data only.

**** Audio leakage example ****

Suppose one original 10-second recording produces 500 feature frames. If you randomly place 400 of those frames into training and 100 into test, then the model has essentially heard the same microphone, room, performer, gain setting, and noise realization in both datasets. The test performance can look excellent. But the model may have learned:

recording identity

instead of

instrument identity.

The stronger experimental split is often by original recording, speaker, performer, session, subject, or device—whatever nuisance factor must generalize. The deck makes this warning explicit: a neural network cannot rescue a flawed split and may simply learn the leakage more effectively.

Slides 28–30: How to interpret training results

The synthetic data are intentionally well separated, so the MLP can approach nearly perfect classification. The deck explicitly warns students not to overgeneralize from that demonstration. A useful way to show different curve patterns is:

**** Healthy learning ****

Training loss:

↓

Validation loss:

↓

Training accuracy:

↑

Validation accuracy:

↑.

Overfitting Training loss continues:

↓

but validation loss begins:

↑.

Meanwhile,

$$A_{\text{train}} \gg A_{\text{val}}.$$

That suggests the model is learning training-specific detail. Underfitting Both training and validation accuracy stay low. Possible causes:

model too simple,

learning rate inappropriate,

insufficient training,

or simply poor features. Confusion matrix Accuracy answers:

“How many predictions were correct?” A confusion matrix answers:

“Which classes were confused with which?” The deck stresses this distinction. Suppose we obtain:

	Kick	Snare	Cymbal
Kick	45	4	1
Snare	5	40	5
Cymbal	0	3	47

The kick/cymbal distinction looks easy. But kick and snare are occasionally confused. An engineering question follows:

“Which DSP features might overlap between kick and snare?” Possible answer:

“Both may have strong transient energy, so onset-related features alone may not discriminate them. Spectral centroid, spectral envelope, or MFCC structure may be needed.” That's much more informative than quoting one overall accuracy number.

Slide 31: Diagnose the broken loop

The code omits: `optimizer.zero_grad()`

The deck's strong answer is:

“The code may still run, but the update no longer represents the gradient of just the current minibatch.” That distinction matters.

A weak answer:

“It crashes.” Usually it does not.

A better answer:

“Gradients accumulate.”

Excellent answer:

“Because PyTorch accumulates `.grad`, each minibatch's gradient is added to previous minibatch gradients. Therefore the optimizer update reflects stale plus current sensitivity unless accumulation is deliberately intended.”

Homework 1: Shapes

Given:

$$N = 64,$$

$$D_{\text{in}} = 8,$$

and

$$D_{\text{hidden}} = 16,$$

the input is

$$\boxed{(64,8)}.$$

The weight matrix is

$$\boxed{(16,8)}.$$

The bias is

$$\boxed{(16,)}.$$

The hidden output is

$$(64,16).$$

These are the instructor answers in the deck.

Homework 2: Why ReLU?

The full derivation should be

$$\mathbf{h} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1,$$

$$\mathbf{y} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2,$$

so

$$\mathbf{y} = \mathbf{W}_2(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \mathbf{W}_2 \mathbf{W}_1 \mathbf{x} + \mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2.$$

Therefore,

$$\mathbf{y} = \mathbf{W}_{\text{eq}} \mathbf{x} + \mathbf{b}_{\text{eq}}.$$

Good explanation:

“Without a nonlinear activation, depth does not increase the class of functions beyond affine maps. ReLU introduces piecewise-affine behavior and allows nonlinear decision regions.”

Homework 3: ReLU

For

$$\mathbf{z} = [-2, -0.5, 0, 1.2, 4]^T,$$

$$\text{ReLU}(\mathbf{z}) = [0, 0, 0, 1.2, 4]^T.$$

For

$$z < 0,$$

$$\text{ReLU}'(z) = 0.$$

For

$$z > 0,$$

$$\text{ReLU}'(z) = 1.$$

At $z = 0$, use the chosen subgradient convention. Homework 4: Cross-entropy For

$$\mathbf{z} = [2, 1, 0],$$

$$p_0 = \frac{e^2}{e^2 + e + 1} \approx 0.6652.$$

Thus,

$$L = -\ln(0.6652) \approx 0.4076.$$

This matches the deck's solution.

Homework 5: Update

Given

$$\theta = 1.5,$$

$$\eta = 0.02,$$

and

$$\frac{\partial L}{\partial \theta} = 4,$$

then

$$\theta_{\text{next}} = 1.5 - 0.02(4) = \boxed{1.42}.$$

Homework 6: Five-step loop

A strong answer should contain both syntax and conceptual purpose.

1. Forward

Compute logits from current parameters.

2. Loss

Reduce prediction error to a scalar objective.

3. Clear

Remove stale gradients.

4. Backward

Compute

$$\nabla_{\theta} L.$$

5. Step

Use the optimizer to modify

$$\theta.$$

If `zero_grad()` is omitted, gradients accumulate across minibatches.

Homework 7: Why scaling matters

A good answer:

“An MLP is trained through gradient-based optimization. If one feature has values around 5000 and another around 0.05, the optimization geometry can be poorly conditioned. Standardization puts features on comparable numerical scales and often leads to more stable and efficient learning.”

A stronger answer adds:

“The scaler must be fit only on training data; otherwise validation/test statistics leak into the model.”

Quiz Q1

Batch size:

32.

Features per example:

8.

Input shape:

`(32,8)`.

Quiz Q2

Why do two affine layers collapse?

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2).$$

Therefore they are equivalent to one affine transformation.

Good answer:

“Composition of affine transformations remains affine; without an activation, adding depth cannot create nonlinear decision boundaries.”

The instructor appendix gives essentially this answer.

Quiz Q3

Should the network apply softmax before CrossEntropyLoss?

No.

Good answer:

“Return raw logits. CrossEntropyLoss internally performs a numerically stable log-softmax plus negative log-likelihood.”

That is exactly the deck's answer.

Quiz Q4

Given

$$p_y = 0.8,$$

$$L = -\ln(0.8)$$

so

$$L \approx 0.223.$$

Quiz Q5

What does `loss.backward()` do?

Weak answer:

“It trains the model.”

Not precise enough.

Good answer:

“It performs reverse-mode automatic differentiation through the forward computational graph and computes the gradient of the scalar loss with respect to upstream learnable parameters.”

Mathematically,

$$\nabla_{\theta} L.$$

The result is stored in parameter .grad tensors.

Quiz Q6

Difference between backward() and step():

$$\text{backward()} = \text{compute gradients}$$

whereas

$$\text{step()} = \text{update parameters using those gradients.}$$

This distinction is central to the whole lecture.

One especially useful connection back to Lecture 25

In Lecture 25, students learned:

$$L \rightarrow \nabla_{\theta} L.$$

But θ was something simple, such as FIR taps. Now,

$$\theta = \{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \mathbf{W}_3, \mathbf{b}_3\}.$$

Nothing fundamental changed about autograd. The graph just got bigger.

That is an important demystification:

A neural network is not using a different kind of calculus. It is simply a larger composition of differentiable tensor operations. One connection to classical classifiers from Week 12 This comparison is worth making explicitly.

A linear SVM forms something like

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b.$$

An MLP begins with the same kind of affine transformation:

$$\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1.$$

But then it adds

$$\text{ReLU}$$

and additional learned affine transformations. So conceptually:

linear classifier + learned nonlinear feature transformations = MLP.

This is a very useful bridge for students who already understand SVMs. What I would expect a senior student to understand By the end of the lecture, a strong senior should be able to explain:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b},$$

why ReLU is necessary,

why the last layer has three logits,

why those logits should not be softmaxed before CrossEntropyLoss,

what cross-entropy measures,

and the five-step loop:

forward → loss → zero → backward → step.

What I would expect a first-year graduate student to understand In addition, I would want the graduate student to be able to derive:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \mathbf{W}_{\text{eq}}\mathbf{x} + \mathbf{b}_{\text{eq}},$$

explain why ReLU creates piecewise-affine regions, derive or at least interpret

$$\frac{\partial L}{\partial z_k} = p_k - \mathbb{1}[k = y],$$

understand the distinction between

$$\nabla_{\theta} L$$

and the optimizer update, and explain why experimental design—particularly scaling and leakage control—is just as important for an MLP as for SVMs or random forests.

The one-minute summary I would want from students

A very good end-of-class answer would be:

“An MLP takes engineered DSP features and repeatedly applies learned affine transformations and nonlinear activations. The final layer produces class logits. Cross-entropy turns those logits and the true class into a differentiable scalar loss. `loss.backward()` computes the gradient of that loss with respect to all trainable weights and biases, and Adam uses those gradients in `optimizer.step()` to update the network. Repeating this over minibatches trains a nonlinear classifier.” That captures the entire lecture and matches the synthesis at the end of the deck: affine layers plus nonlinear activations create flexible decision functions, cross-entropy supplies the scalar objective,

autograd computes the gradients, and Adam turns those gradients into parameter updates.